

Módulos de Software Stand-alone, Web y Móvil Codificados y Probados

Jhon Ramiro Zapata Pérez

Instructor

Andrés Montoya

Centro de la Tecnología del Diseño y la Productividad Empresarial Regional

Cundinamarca sede Girardot

Análisis y Desarrollo de Software

Julio de 2026

Caloto-Cauca

Introducción

El presente documento describe el funcionamiento de una API REST desarrollada en Python utilizando el framework FastAPI. La API implementa un sistema básico de gestión de productos, permitiendo realizar las operaciones fundamentales de Creacion, consulta, actualización y eliminación de información (conocidas comúnmente como operaciones CRUD).

A lo largo del documento se explicará el código fuente de la aplicación paso a paso, detallando la función de cada componente, modelo y endpoint. Adicionalmente, se incluye evidencia practica del funcionamiento de la API mediante pruebas realizadas con la herramienta Postman, en las cuales se muestran las solicitudes enviadas y las respuestas obtenidas para cada una de estas funcionalidades implementadas.

Objetivo

Explicar de manera clara y estructurada el desarrollo de una API REST construida con FastAPI, describiendo:

- La estructura general del proyecto y las tecnologías utilizadas.
- EL propósito y funcionamiento de cada endpoint implementado.
- La lógica interna del código, incluyendo el modelo de datos y el manejo de errores.
- La evidencia de las pruebas funcionales realizadas sobre cada endpoint mediante Postman.

Descripción general de la API

La API fue construida utilizando FastAPI, un framework moderno de Python orientado a la creación de servicios web rápidos y con documentación automática. Para efectos de este ejemplo, los datos de los productos se almacenan en una lista en memoria (para este ejemplo no se utiliza una base de datos), por lo que la información se reinicia cada vez que el servidor se reinicia.

Framework	FastAPI
Lenguaje	Python
Servidor	Uvicorn
Puerto	8000
Host local	http://127.0.0.1:8000
Persistencia	En memoria (lista de Python)

A continuación, se muestra el código fuente completo de la aplicación (main.py)

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
from typing import Optional, List

# Instancia principal de la aplicación FastAPI
```

```

app = FastAPI(title="API Básica de Ejemplo", version="1.0")

# Modelo para recibir datos (se usa al crear o actualizar productos)
class Producto(BaseModel):
    nombre: str
    precio: float
    descripcion: Optional[str] = None

# Lista en memoria (solo para este ejemplo, en un caso real esto sería una base de datos)
productos = [
    {"id": 1, "nombre": "Laptop", "precio": 899.99, "descripcion": "Laptop gaming"},
    {"id": 2, "nombre": "Mouse", "precio": 29.99, "descripcion": "Mouse inalámbrico"}
]

# Contador para asignar el próximo id disponible.
# Se calcula a partir del id más alto existente para evitar duplicados
# si en algún momento se elimina un producto.
siguiente_id = max((p["id"] for p in productos), default=0) + 1

@app.get("/")
def home():
    return {"mensaje": "¡Bienvenido a mi API básica en Python!"}

# READ (listar todos los productos)
@app.get("/productos", response_model=List[dict])
def obtener_productos():
    return productos

# READ (obtener un solo producto por su id)
@app.get("/productos/{producto_id}")

```

```

def obtener_producto(producto_id: int):
    for producto in productos:
        if producto["id"] == producto_id:
            return producto

    # Si no se encuentra, se responde con un 404 real en vez de un 200 con "error"
    raise HTTPException(status_code=404, detail="Producto no encontrado")

# CREATE (agregar un nuevo producto)
@app.post("/productos")
def crear_producto(producto: Producto):
    global siguiente_id
    nuevo_producto = {
        "id": siguiente_id,
        "nombre": producto.nombre,
        "precio": producto.precio,
        "descripcion": producto.descripcion
    }
    productos.append(nuevo_producto)
    siguiente_id += 1
    return {"mensaje": "Producto creado correctamente", "producto": nuevo_producto}

# UPDATE (modificar un producto existente por su id)
@app.put("/productos/{producto_id}")
def actualizar_producto(producto_id: int, producto: Producto):
    for p in productos:
        if p["id"] == producto_id:
            # Se actualizan los campos manteniendo el mismo id
            p["nombre"] = producto.nombre
            p["precio"] = producto.precio
            p["descripcion"] = producto.descripcion

```

```

        return {"mensaje": "Producto actualizado correctamente", "producto": p}
    raise HTTPException(status_code=404, detail="Producto no encontrado")

# DELETE (eliminar un producto por su id)
@app.delete("/productos/{producto_id}")
def eliminar_producto(producto_id: int):
    for p in productos:
        if p["id"] == producto_id:
            productos.remove(p)
            return {"mensaje": "Producto eliminado correctamente"}
    raise HTTPException(status_code=404, detail="Producto no encontrado")

@app.get("/saludo/{nombre}")
def saludar(nombre: str):
    return {"saludo": f"!Hola {nombre}!, ¿cómo estás?"}

if __name__ == "__main__":
    import uvicorn
    uvicorn.run(
        "main:app",
        host="127.0.0.1", # Cambia a "0.0.0.0" si quieres acceder desde el móvil
        port=8000,
        reload=True
    )

```

Explicación del código paso a paso

- **Importaciones y configuración inicial**

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
from typing import Optional, List

# Instancia principal de la aplicación FastAPI
app = FastAPI(title="API Básica de Ejemplo", version="1.0")
```

Se importan las herramientas principales: FastAPI para crear la aplicación web y manejar las excepciones HTTP, y BaseModel de Pydantic para definir y validar automáticamente la estructura de los datos recibidos. Se crea la instancia principal de la aplicación (app), que es el objeto sobre el cual se registran todas las rutas o endpoints.

- **Modelo de datos (Producto)**

```
# Modelo para recibir datos (se usa al crear o actualizar productos)
class Producto(BaseModel):
    nombre: str
    precio: float
    descripcion: Optional[str] = None
```

Este modelo define la estructura que debe tener la información enviada al crear o actualizar un producto. FastAPI utiliza esta clase para validar automáticamente que “nombre” sea texto, “precio” sea numérico y

“descripción” sea un campo de texto opcional. Si el cliente envía datos con un formato incorrecto, la API responde automáticamente con un error de validación.

- **Almacenamiento en memoria**

```
# Lista en memoria (solo para este ejemplo, en un caso real esto sería una base de datos)
productos = [
    {"id": 1, "nombre": "Laptop", "precio": 899.99, "descripcion": "Laptop gaming"},
    {"id": 2, "nombre": "Mouse", "precio": 29.99, "descripcion": "Mouse inalámbrico"}
]

# Contador para asignar el próximo id disponible.
# Se calcula a partir del id más alto existente para evitar duplicados
# si en algún momento se elimina un producto.
siguiente_id = max((p["id"] for p in productos), default=0) + 1
```

Ante la ausencia de una base de datos, se utiliza una lista de Python como almacenamiento temporal, precargada con dos productos de ejemplo. La variable “siguiente_id” calcula automáticamente el próximo identificador disponible tomando el id mas alto existente y sumándole uno, lo que evita que se generen ids duplicados incluso si se elimina algún producto previamente.

- **Endpoint raíz – GET/**

```
@app.get("/")
def home():
    return {"mensaje": "¡Bienvenido a mi API básica en Python!"}
```

Ruta de bienvenida que confirma que el servicio esta activo y respondiendo correctamente.

- **Listar productos – GET/productos**


```
# READ (listar todos los productos)
@app.get("/productos", response_model=List[dict])
def obtener_productos():
    return productos
```

Devuelve la lista completa de productos almacenados actualmente en memoria.

- **Obtener un producto por id – GET/productos/{producto_id}**

```
# READ (obtener un solo producto por su id)
@app.get("/productos/{producto_id}")
def obtener_producto(producto_id: int):
    for producto in productos:
        if producto["id"] == producto_id:
            return producto
    # Si no se encuentra, se responde con un 404 real en vez de un 200 con "error"
    raise HTTPException(status_code=404, detail="Producto no encontrado")
```

Recorre la lista de productos buscando uno cuyo id coincida con el parámetro recibido en la URL. Si lo encuentra, lo retorna; si no, se lanza una excepción `HTTPException` con código 404, devolviendo un error real en lugar de una respuesta 200 con un mensaje de error, lo cual es una buena practica en el diseño de APIs REST.

- **Crear un producto – POST/productos**

```
# CREATE (agregar un nuevo producto)
@app.post("/productos")
def crear_producto(producto: Producto):
    global siguiente_id
    nuevo_producto = {
        "id": siguiente_id,
        "nombre": producto.nombre,
        "precio": producto.precio,
        "descripcion": producto.descripcion
    }
    productos.append(nuevo_producto)
    siguiente_id += 1
    return {"mensaje": "Producto creado correctamente", "producto": nuevo_producto}
```

Recibe un objeto producto validado por Pydantic, le asigna automáticamente el siguiente id disponible (ignorando cualquier id que el cliente intente enviar), lo agrega a la lista y retorna un mensaje de confirmación junto con el producto creado.

- **Actualizar un producto – PUT/productos/{producto_id}**

```
# UPDATE (modificar un producto existente por su id)
@app.put("/productos/{producto_id}")
def actualizar_producto(producto_id: int, producto: Producto):
    for p in productos:
        if p["id"] == producto_id:
            # Se actualizan los campos manteniendo el mismo id
            p["nombre"] = producto.nombre
            p["precio"] = producto.precio
            p["descripcion"] = producto.descripcion
            return {"mensaje": "Producto actualizado correctamente", "producto": p}
    raise HTTPException(status_code=404, detail="Producto no encontrado")
```

Busca el producto por id y, si existe, actualiza sus campos (nombre, precio, descripción) conservando el mismo identificador. Si el id no existe, retorna en un error 404.

- **Eliminar un producto – DELETE/productos/{producto_id}**

```
# DELETE (eliminar un producto por su id)
@app.delete("/productos/{producto_id}")
def eliminar_producto(producto_id: int):
    for p in productos:
        if p["id"] == producto_id:
            productos.remove(p)
            return {"mensaje": "Producto eliminado correctamente"}
    raise HTTPException(status_code=404, detail="Producto no encontrado")
```

Elimina de la lista el producto cuyo id coincide con el recibido. Si no se encuentra, retorna un error 404.

- **Endpoint de saludo – GET/saludo/{nombre}**

```
@app.get("/saludo/{nombre}")
def saludar(nombre: str):
    return {"saludo": f"¡Hola {nombre}!, ¿cómo estás?"}
```

Endpoint adicional que recibe un nombre como parámetro de ruta y devuelve un saludo personalizado.

- **Punto de entrada de la aplicación**

```
if __name__ == "__main__":
    import uvicorn
    uvicorn.run(
        "main:app",
        host="127.0.0.1", # Cambia a "0.0.0.0" si quieres acceder desde el móvil
        port=8000,
        reload=True
    )
```

Permite ejecutar la aplicación directamente con Python, levantando el servidor Uvicorn en el puerto 8000 con recarga automática activada, útil durante el desarrollo.

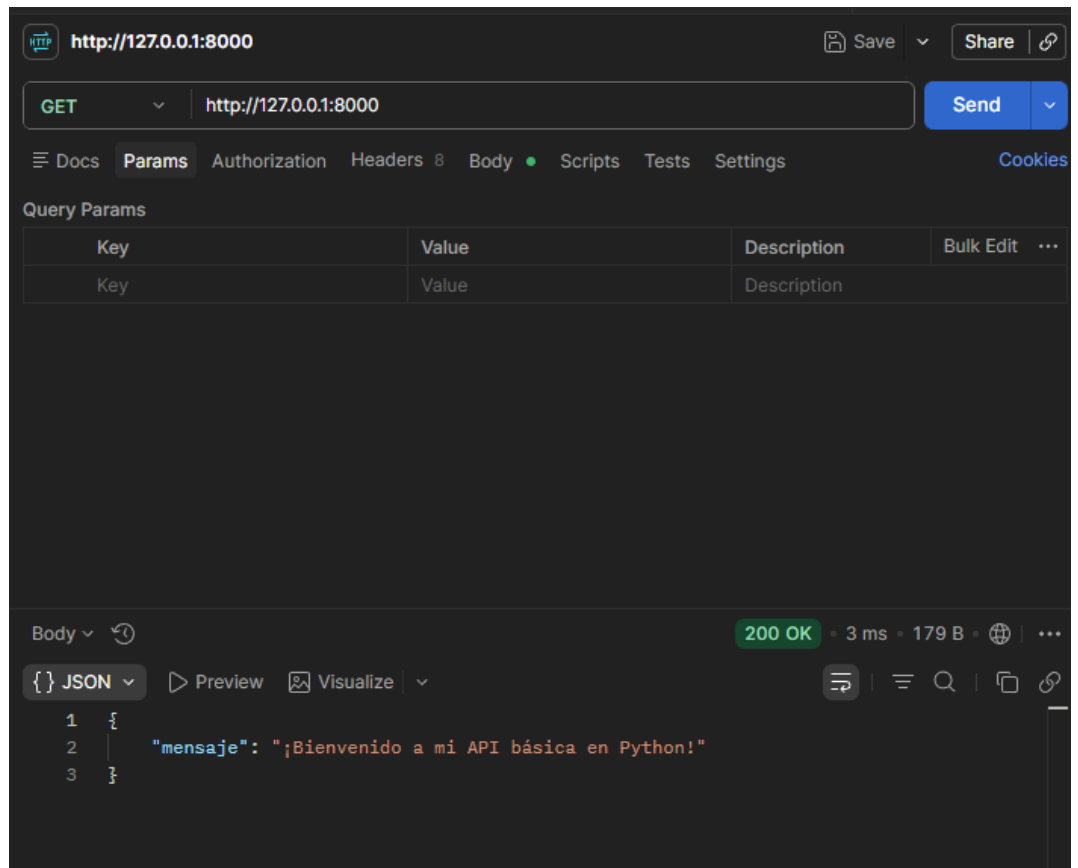
Pruebas funcionales en Postman

A continuación, se presenta la evidencia de las pruebas realizadas sobre cada uno de los endpoints de la API, utilizando la herramienta Postman. Para cada prueba se muestra el método HTTP, la URL utilizada y la captura de pantalla correspondiente con la respuesta obtenida del servidor.

- **GET/ - Endpoint de bienvenida**

Se realiza una solicitud GET a la raíz del servicio para verificar que la API está activa.

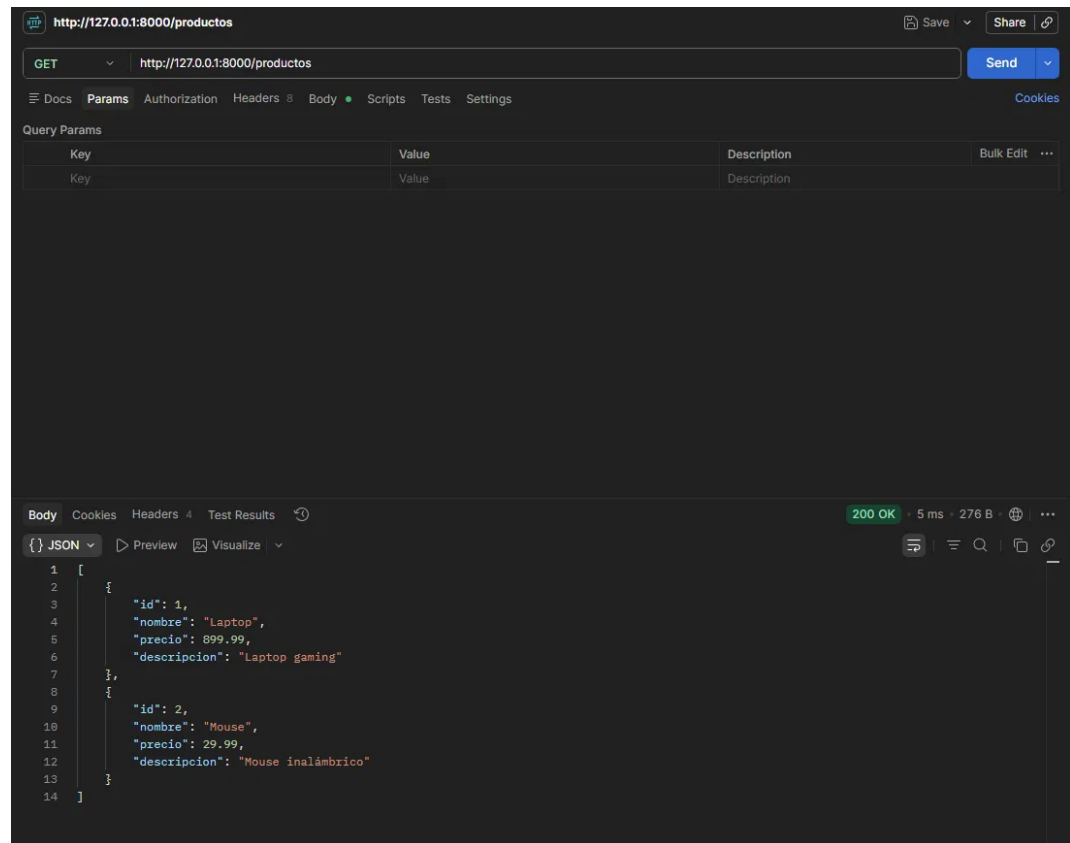
Método	GET
URL	http://127.0.0.1:8000/



- **GET/productos – Listar todos los productos**

Se consulta la lista completa de productos registrados en memoria.

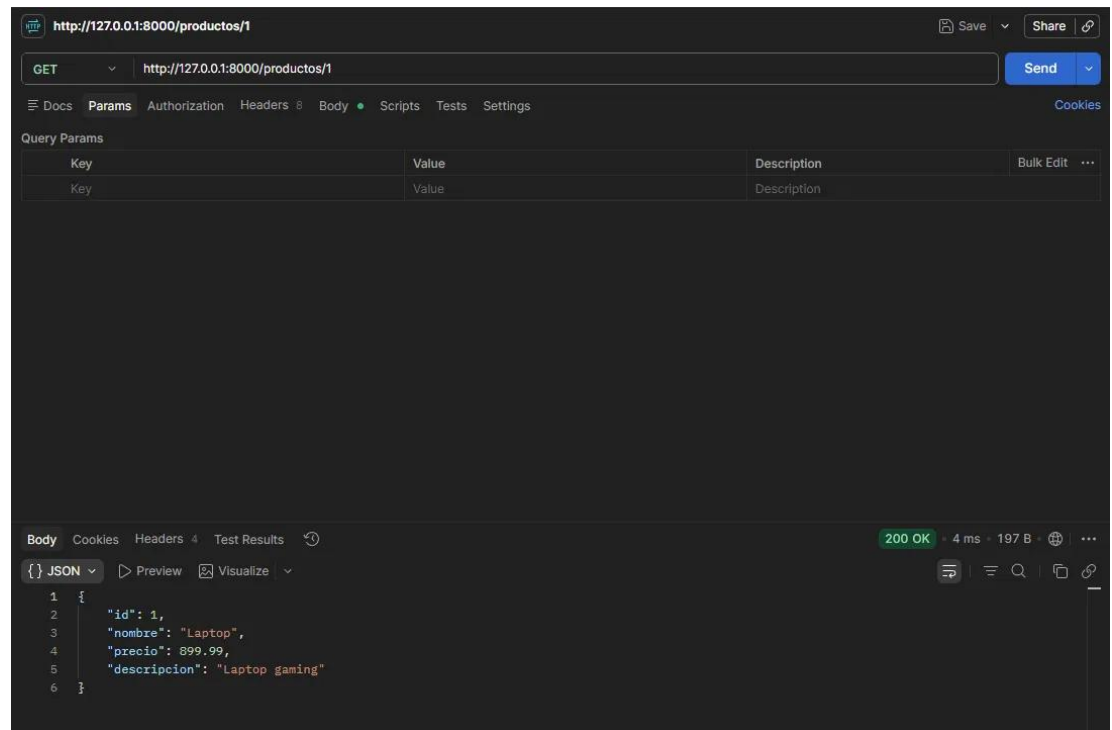
Método	GET
URL	http://127.0.0.1:8000/productos



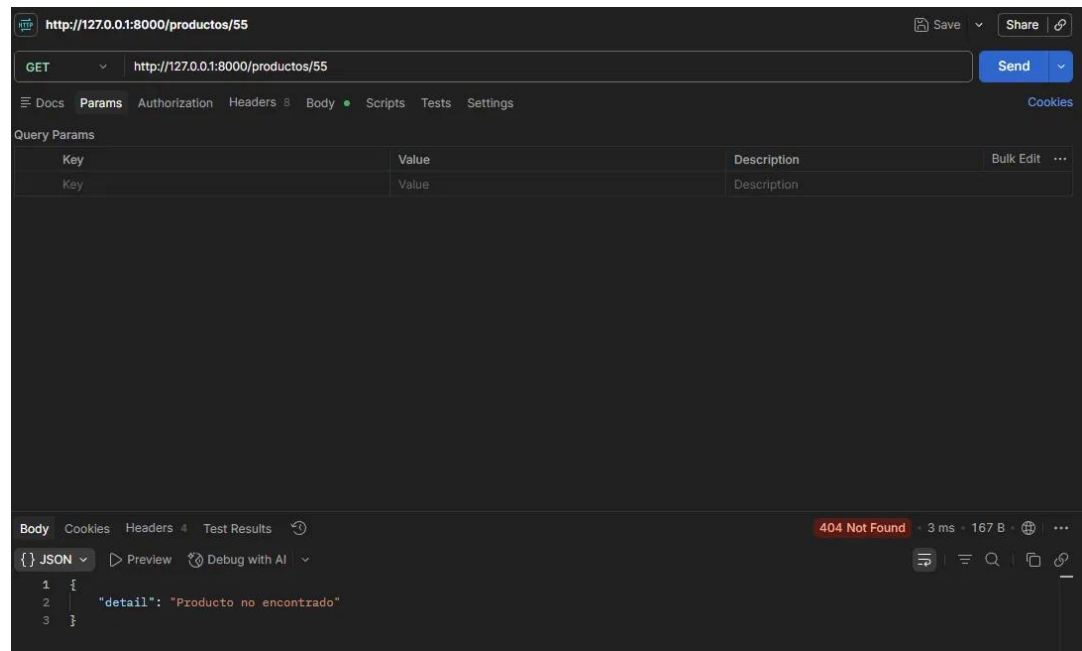
- **GET/productos/{id} – Obtener un producto específico**

Se prueba tanto el caso exitoso (id existente) como el caso de error (id inexistente).

Método	GET
URL	http://127.0.0.1:8000/productos/1



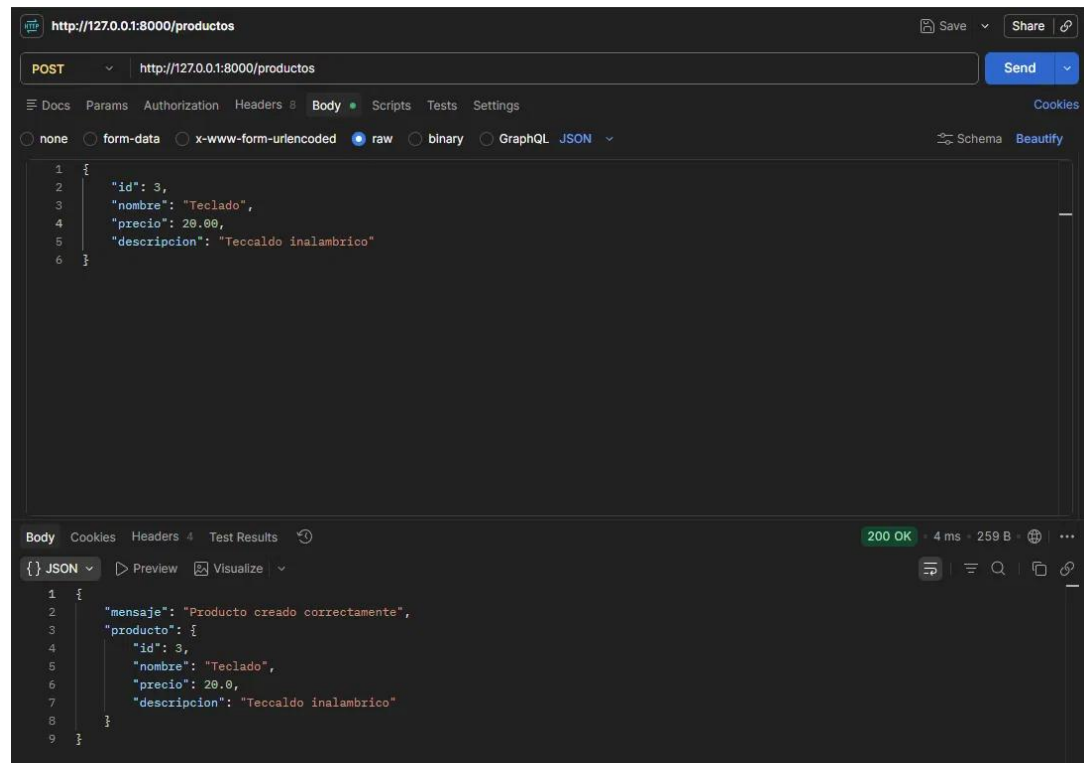
Método	GET
URL	http://127.0.0.1:8000/productos/55



- **POST/productos – Crear un nuevo producto**

Se envía un nuevo producto en el cuerpo de la solicitud; el servidor le asigna automáticamente el id 3.

Método	POST
URL	http://127.0.0.1:8000/productos



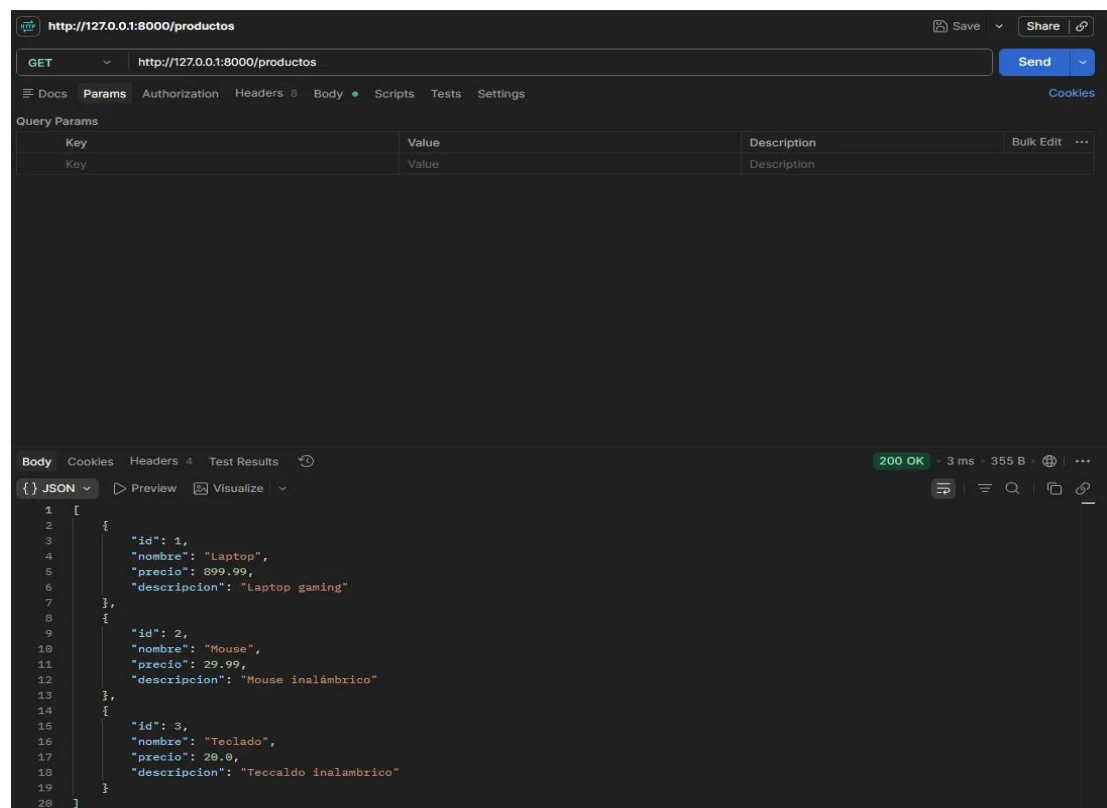
```
http://127.0.0.1:8000/productos

POST http://127.0.0.1:8000/productos

Body
1 {
2   "id": 3,
3   "nombre": "Teclado",
4   "precio": 20.00,
5   "descripcion": "Teclado inalámbrico"
6 }

Body
1 {
2   "mensaje": "Producto creado correctamente",
3   "producto": {
4     "id": 3,
5     "nombre": "Teclado",
6     "precio": 20.0,
7     "descripcion": "Teclado inalámbrico"
8   }
9 }
```

Se verifica el resultado consultando nuevamente el listado completo:



```
http://127.0.0.1:8000/productos

GET http://127.0.0.1:8000/productos

Query Params


| Key | Value | Description | Bulk Edit |
|-----|-------|-------------|-----------|
| Key | Value | Description |           |

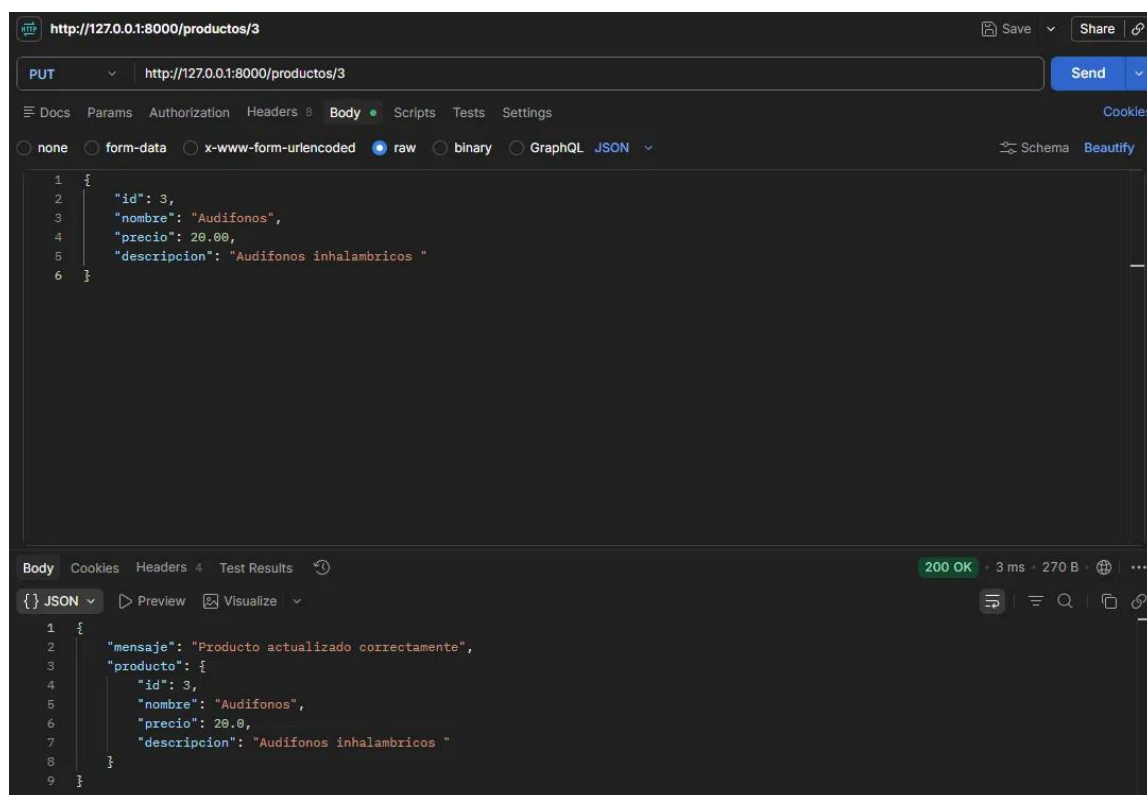


Body
1 [
2   {
3     "id": 1,
4     "nombre": "Laptop",
5     "precio": 899.99,
6     "descripcion": "Laptop gaming"
7   },
8   {
9     "id": 2,
10    "nombre": "Mouse",
11    "precio": 29.99,
12    "descripcion": "Mouse inalámbrico"
13  },
14  {
15    "id": 3,
16    "nombre": "Teclado",
17    "precio": 20.0,
18    "descripcion": "Teclado inalámbrico"
19  }
20 ]
```

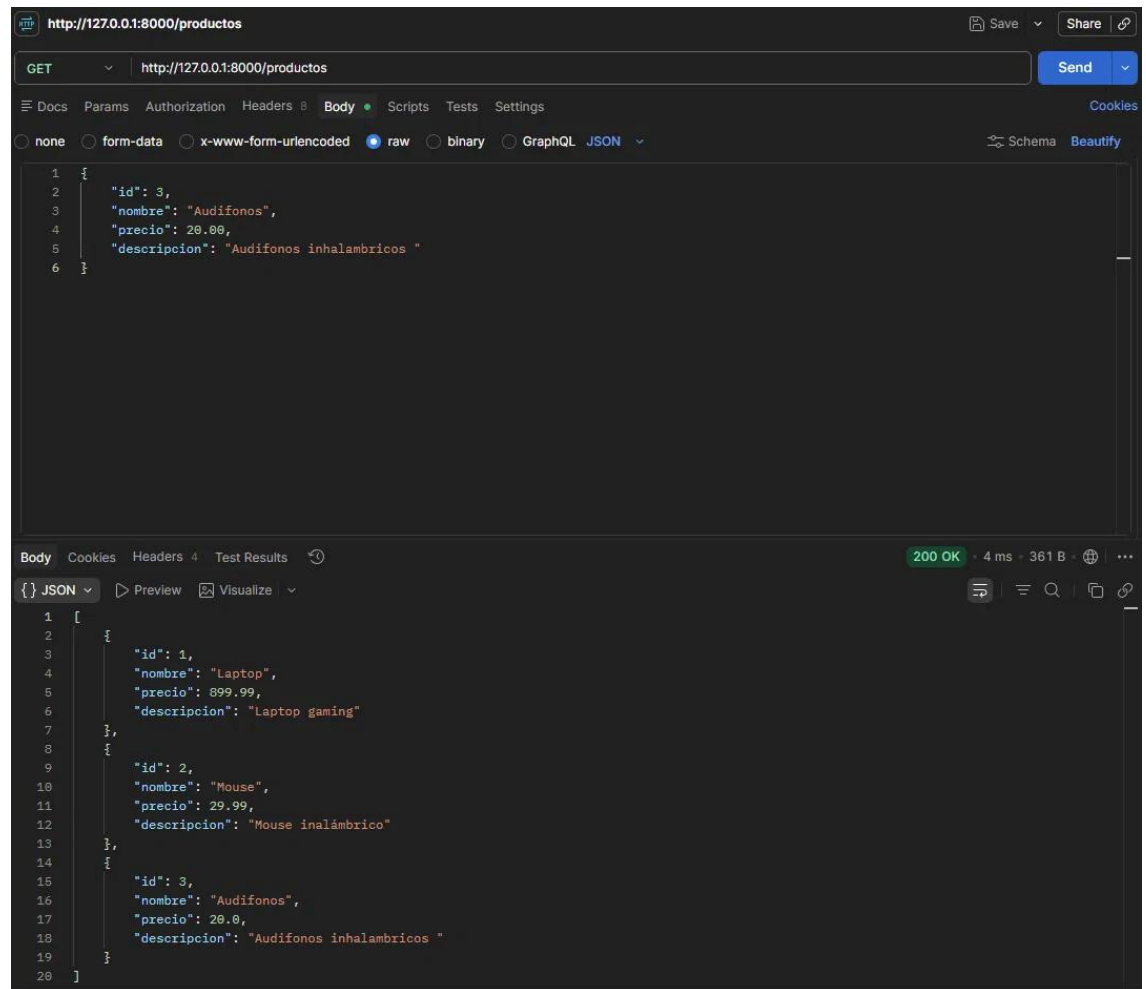
- **PUT/productos/{id} – Actualizar un producto**

Se actualizan los datos del producto con id 3, cambiando su nombre y descripción.

Método	PUT
URL	http://127.0.0.1:8000/productos/3



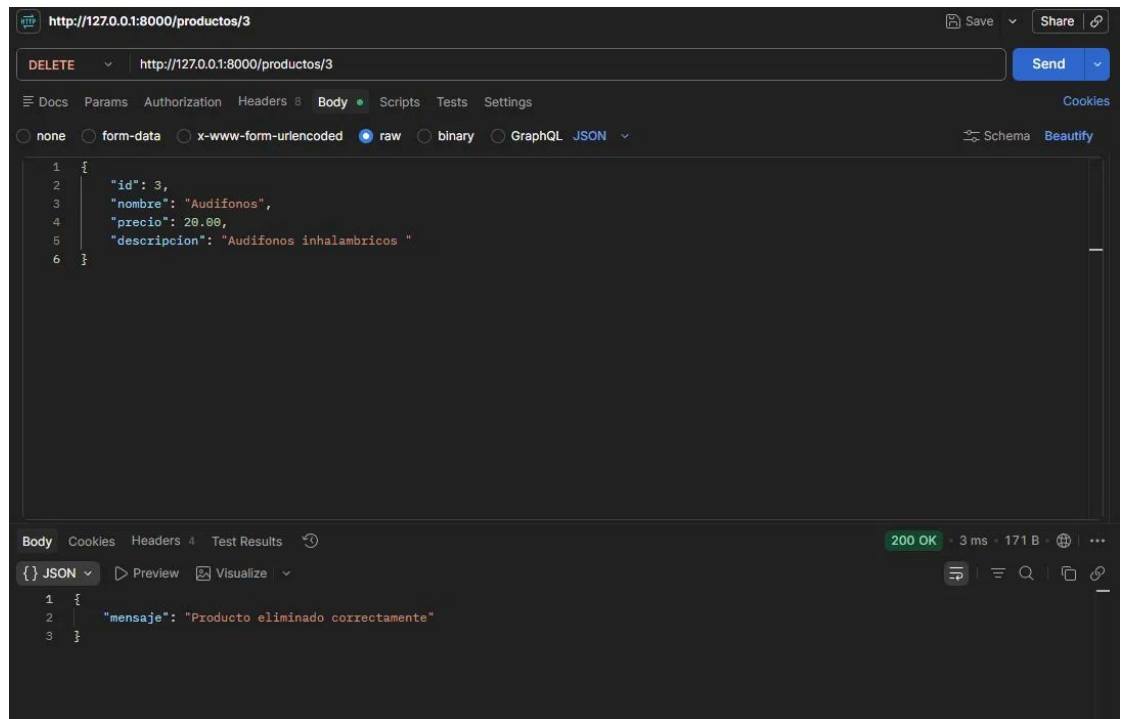
Verificación del cambio en el listado general:



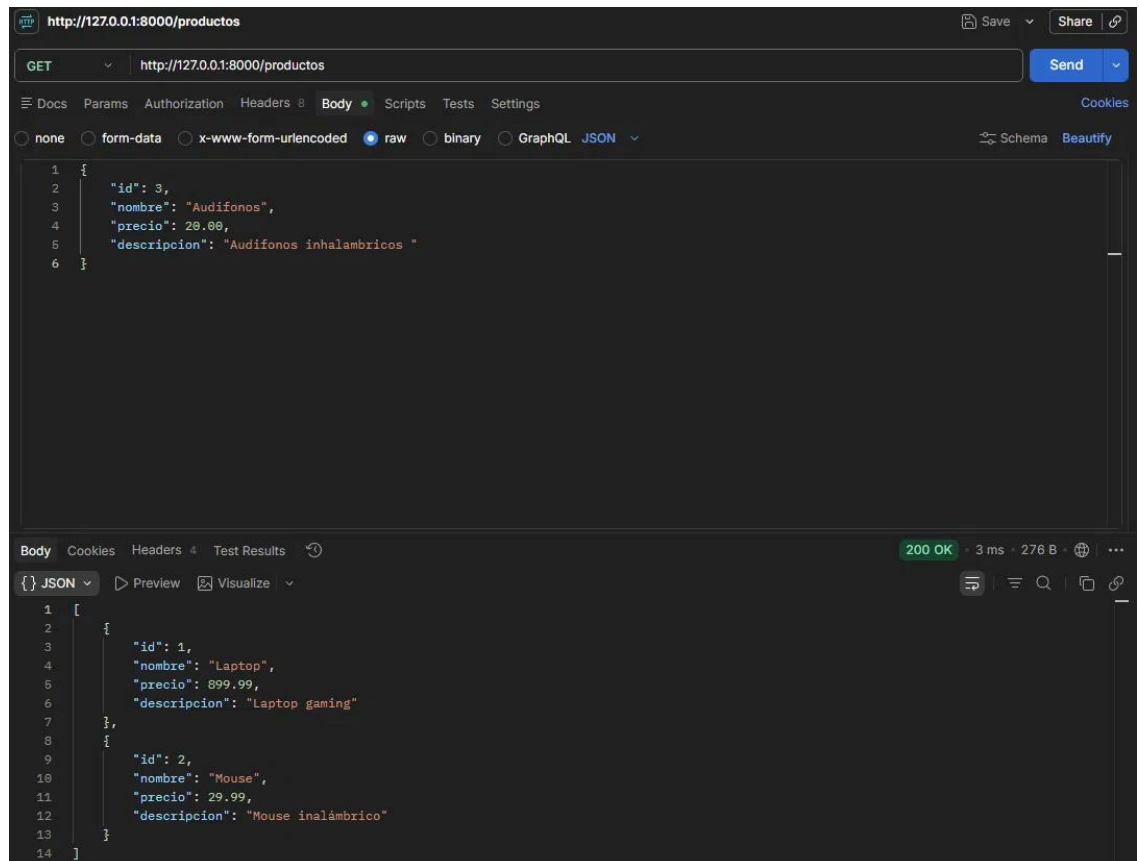
- **DELETE/productos/{id} – Eliminar un producto**

Se elimina el producto con id 3 previamente creado y actualizado.

Método	DELETE
URL	<code>http://127.0.0.1:8000/productos/3</code>



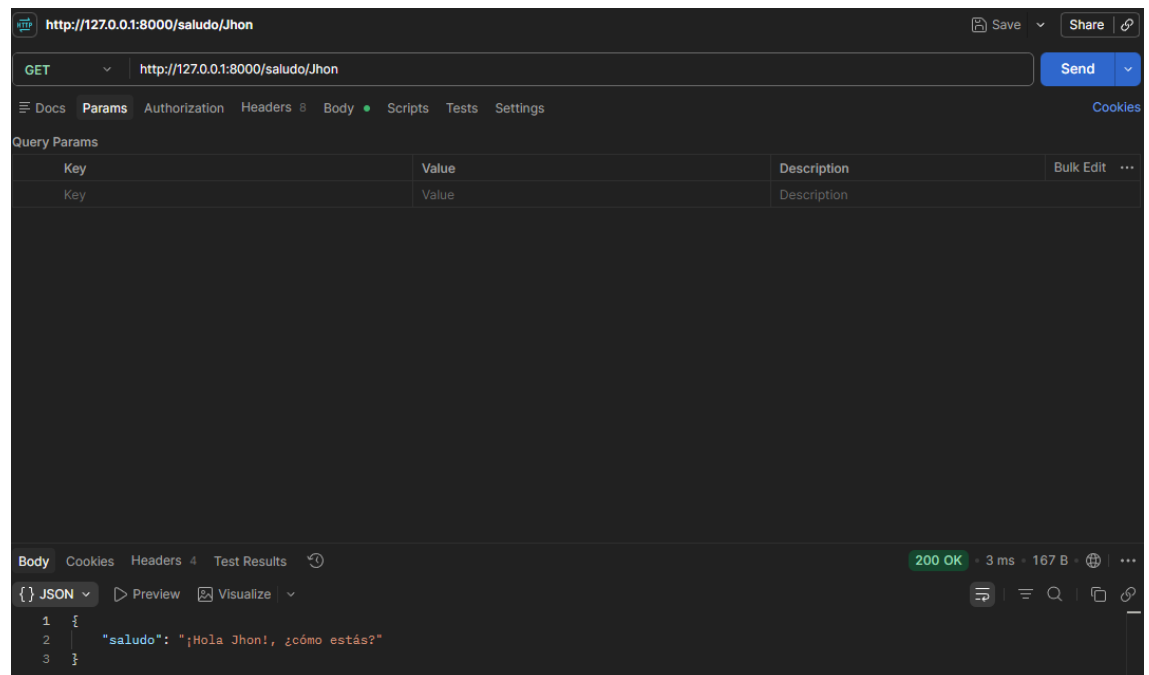
Verificación final del listado, confirmando que el producto ya no existe:



- **GET/saludo/{nombre} – Saludo personalizado**

Se consulta el endpoint de saludo enviando un nombre como parámetro de ruta.

Método	GET
URL	http://127.0.0.1:8000/saludo/Jhon



Conclusiones

- La API desarrollada con FastAPI cumple correctamente con las operaciones CRUD básicas (crear, leer, actualizar y eliminar) sobre un recurso de tipo producto.
- El uso de Pydantic para la validación de datos garantiza que la información recibida en las solicitudes POST y PUT cumpla con la estructura esperada antes de ser procesada.

- El manejo explícito de errores mediante `HTTPException` permite que la API responda con códigos de estado HTTP apropiados (por ejemplo, 404 cuando un producto no existe), lo cual mejora la claridad y la robustez del servicio frente a un cliente que lo consume.
- Las pruebas realizadas en Postman evidencian que todos los endpoints funcionan correctamente bajo condiciones normales y de error, validando así el comportamiento esperado del sistema.
- Al tratarse de un almacenamiento en memoria, la principal limitación de esta API es la pérdida de los datos al reiniciar el servidor; una mejora futura consistiría en integrar una base de datos persistente (por ejemplo, SQLite o PostgreSQL).
- En general, el proyecto constituye una base sólida y didáctica para comprender los fundamentos de la construcción de servicios REST con FastAPI.